

PASSWORDLESS AUTHENTICATION

PEM Key Access, Password Configuration,
and Ansible Playbook Automation
for AWS EC2 Instances

Professional Reference Documentation

Version 1.0

Table of Contents

1. Introduction to Passwordless Authentication
2. Authentication Methods: A Security Comparison
 - 2.1 Password Authentication
 - 2.2 Public Key Authentication
 - 2.3 PEM Files in AWS
3. Connecting to EC2 with a PEM File
 - 3.1 Obtaining the PEM File
 - 3.2 Setting File Permissions
 - 3.3 Connecting via SSH
 - 3.4 Common Errors
4. Password-Based Authentication on EC2
 - 4.1 Default AWS Configuration
 - 4.2 Enabling Password Authentication
 - 4.3 Setting User Passwords
 - 4.4 Security Warnings
5. Passwordless Authentication via Ansible Playbook
 - 5.1 Architecture Overview
 - 5.2 Generating the Control Node SSH Key
 - 5.3 The Ansible Inventory
 - 5.4 The Playbook
 - 5.5 Running the Playbook
 - 5.6 Verification
6. Hardening: Disabling Password Authentication
7. Best Practices
8. Quick Reference

1. Introduction to Passwordless Authentication

Passwordless authentication is an access model in which a user or automated system proves its identity using a cryptographic key pair rather than a shared secret (password). The user holds a private key; the server holds the corresponding public key. When a connection is attempted, the server issues a challenge that can only be answered correctly by the holder of the private key. No password ever crosses the network.

In the context of AWS EC2, passwordless authentication takes three practical forms:

- **PEM file authentication** — the standard AWS mechanism, where the private key downloaded at instance launch is used with `ssh -i` to connect.
- **Password-based authentication** — the traditional Unix SSH model, disabled by default on EC2 but sometimes required for legacy tooling or third-party clients.
- **Ansible-managed key deployment** — an automation pattern in which a playbook distributes a control node's public key to managed instances, establishing passwordless access for all future automation.

This document covers all three, with a strong emphasis on the Ansible workflow, which is the standard approach in production environments.

Why passwordless matters:

- **Eliminates brute-force attacks.** A private key cannot be guessed; an attacker would need to steal the key file.
- **Enables automation.** Scripts, CI/CD pipelines, and configuration management tools cannot type a password interactively.
- **Scales cleanly.** A single public key can be distributed to hundreds of instances by an Ansible playbook.
- **Supports auditing.** Key issuance and revocation are explicit events, unlike password changes that are often invisible.

2. Authentication Methods: A Security Comparison

2.1 Password Authentication

Password authentication requires the user to transmit a shared secret over the encrypted SSH channel. The server compares the received password against a stored hash. While SSH encrypts the transmission, passwords remain vulnerable to:

- Brute-force and dictionary attacks against the SSH daemon.
- Reuse across systems, where one breach compromises many hosts.
- Weak or default credentials left in place.
- Interception if the server is compromised or misconfigured.

Because of these risks, password authentication is disabled by default on Amazon EC2 Linux instances.[reference:0]

2.2 Public Key Authentication

Public key authentication uses asymmetric cryptography. The private key never leaves the client; only a challenge response is sent. The server validates the response using the public key stored in the user's `~/.ssh/authorized_keys` file.

Security advantages over passwords:

- **Nothing secret travels over the wire.** Even if the session were somehow decrypted, no reusable credential is exposed.
- **Immune to guessing.** The key space is astronomically large; brute-forcing a 4096-bit key is computationally infeasible.
- **No password reuse.** Each key is unique to its owner and can be revoked independently.
- **Supports passphrases.** A passphrase-protected private key adds a second factor without weakening the first.

Public key authentication is the recommended method by OpenSSH and by AWS itself.[reference:1]

2.3 PEM Files in AWS

When an EC2 instance is launched, AWS offers to create a key pair or to use an existing one. The private key is downloaded once, in PEM format (Privacy Enhanced Mail). This file is the only copy; AWS does not retain it. The corresponding public key is injected into the instance's `~/.ssh/authorized_keys` file at boot.

Key facts about PEM files:

- **Format.** PEM is a base64-encoded container. PuTTY on Windows requires conversion to PPK format.
- **Permissions.** OpenSSH refuses to use a private key that is readable by anyone other than the owner. Set mode 400.
- **Single download.** If the PEM file is lost, it cannot be re-downloaded. A new key pair must be created.
- **Scope.** The key pair is bound to the region in which it was created; it cannot be used in another region without re-importing the public key.

3. Connecting to EC2 with a PEM File

3.1 Obtaining the PEM File

The PEM file is downloaded when the instance is launched. Its default name matches the key pair name, with a .pem extension. Store it in a secure location, ideally outside the repository tree.

```
# Recommended location on Linux / macOS
mv ~/Downloads/my-key-pair.pem ~/.ssh/my-key-pair.pem

# Recommended location on Windows
# Place in a dedicated folder, e.g. C:\Keys\my-key-pair.pem
```

3.2 Setting File Permissions

OpenSSH will reject a private key that is group- or world-readable. On Linux and macOS, use `chmod 400`; on Windows, use the file properties dialog to remove inherited permissions.[reference:2]

```
# Linux / macOS
chmod 400 ~/.ssh/my-key-pair.pem

# Verify permissions (should show -r-----)
ls -l ~/.ssh/my-key-pair.pem

# Windows PowerShell (alternative to GUI)
icacls C:\Keys\my-key-pair.pem /inheritance:r
icacls C:\Keys\my-key-pair.pem /grant:r "$($env:USERNAME):(R)"
```

3.3 Connecting via SSH

```
# Amazon Linux
ssh -i ~/.ssh/my-key-pair.pem ec2-user@<public-ip>

# Ubuntu
ssh -i ~/.ssh/my-key-pair.pem ubuntu@<public-ip>

# Debian
ssh -i ~/.ssh/my-key-pair.pem admin@<public-ip>

# Rocky Linux
ssh -i ~/.ssh/my-key-pair.pem rocky@<public-ip>

# RHEL
ssh -i ~/.ssh/my-key-pair.pem ec2-user@<public-ip>
```

The default username depends on the AMI. The following table lists common distributions.[reference:3]

Distribution	Default user
Amazon Linux	ec2-user
Ubuntu	ubuntu
Debian	admin

Distribution	Default user
RHEL	ec2-user
CentOS	centos
Fedora	fedora
Rocky Linux	rocky
SUSE	ec2-user

Table 1 — Default SSH user by distribution.

3.4 Common Errors

- **"Permissions are too open."** The PEM file has incorrect permissions. Run `chmod 400`.
- **"Connection timed out."** The security group does not allow inbound SSH (port 22) from your IP address.
- **"Permission denied (publickey)."** The wrong user name or the wrong PEM file. Verify both.
- **"No such file or directory."** The path to the PEM file is incorrect. Use an absolute path.

4. Password-Based Authentication on EC2

4.1 Default AWS Configuration

On all Amazon EC2 Linux instances, password authentication and direct root login are disabled by default. The SSH daemon is configured with `PasswordAuthentication no` and `PermitRootLogin no`. Only key-based authentication is accepted.[reference:4]

This default exists for sound security reasons. Enabling password authentication reintroduces the brute-force and password-reuse risks that key-based authentication eliminates. It should be done only when a specific requirement demands it, and never on internet-facing instances without additional safeguards.

4.2 Enabling Password Authentication

To enable password authentication, edit `/etc/ssh/sshd_config` on the instance. Keep an existing SSH session open in case the change prevents reconnection, and always back up the file first.[reference:5]

```
# Back up the original configuration
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak

# Edit the file
sudo vi /etc/ssh/sshd_config

# Locate the PasswordAuthentication line and set it to yes
# PasswordAuthentication yes

# Optionally, disable key-based authentication (not recommended)
# PubkeyAuthentication no

# Save the file, then restart the SSH daemon
sudo systemctl restart sshd

# Verify the SSH daemon accepted the configuration
sudo sshd -t
```

On Ubuntu, the service is called `ssh` rather than `sshd`: `sudo service ssh restart`.[\[reference:6\]](#)

On AMIs that use cloud-init, the setting can also be controlled through `ssh_pwauth` in `/etc/cloud/cloud.cfg`. Setting it to 1 causes cloud-init to configure `PasswordAuthentication yes` at boot.

4.3 Setting User Passwords

Once password authentication is enabled, each user must have a password set. New users created with `useradd` have no password by default and cannot log in until one is set with `passwd`.

```
# Set a password for an existing user
sudo passwd ec2-user

# Create a new user and set a password
sudo adduser deploy
sudo passwd deploy

# Verify the account is not locked
sudo passwd -S deploy
```

4.4 Security Warnings

- **Never enable password authentication on an internet-facing instance without fail2ban or an equivalent rate limiter.** Exposed SSH daemons with password authentication are scanned and attacked within minutes of becoming reachable.
- **Prefer key-based authentication even when passwords are enabled.** The two can coexist; leave `PubkeyAuthentication yes` and use passwords only as a fallback for specific accounts.
- **Restrict source IPs in the security group.** Limit inbound port 22 to known networks rather than `0.0.0.0/0`.
- **Enforce strong password policies.** Use PAM modules such as `pam_pwquality` to reject weak passwords.
- **Audit authentication attempts.** Monitor `/var/log/auth.log` or `/var/log/secure` for failed login attempts.

5. Passwordless Authentication via Ansible Playbook

The most scalable way to establish passwordless authentication across many EC2 instances is an Ansible playbook. The playbook uses the initial PEM key to connect once, then distributes the control node's public SSH key to each instance. From that point forward, the PEM file is no longer needed for routine automation.[reference:7]

5.1 Architecture Overview

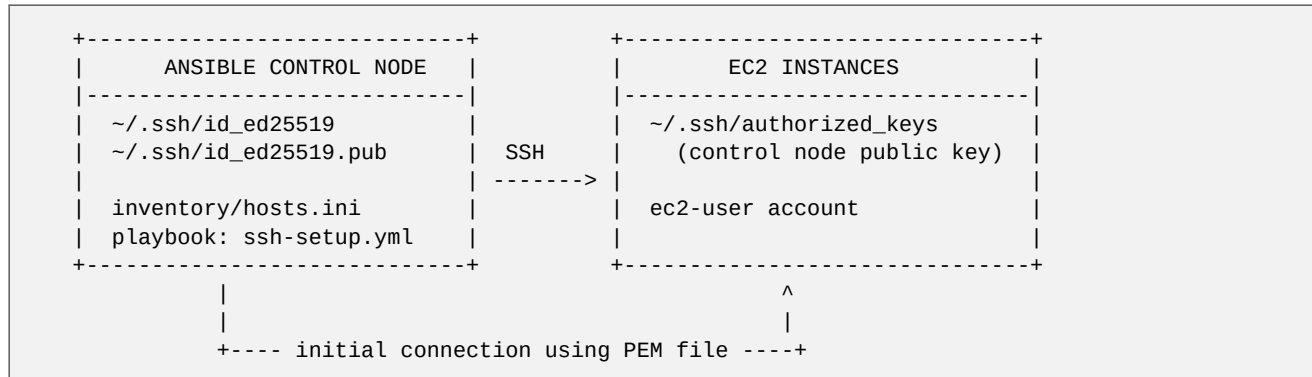


Figure 1 — Control node distributes its public key to EC2 instances.

5.2 Generating the Control Node SSH Key

If the control node does not already have an SSH key pair, generate one. Ed25519 is preferred for its strong security and short key length.

```

# Generate an Ed25519 key pair (recommended)
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -C "ansible-control"

# Or RSA 4096 if Ed25519 is not supported by a legacy target
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -C "ansible-control"

# Verify the key pair
ls -l ~/.ssh/id_ed25519*
cat ~/.ssh/id_ed25519.pub

```

5.3 The Ansible Inventory

```

# inventory/hosts.ini
[ec2_instances]
web01 ansible_host=192.0.2.10
web02 ansible_host=192.0.2.11
db01  ansible_host=192.0.2.20

[ec2_instances:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/my-key-pair.pem
ansible_python_interpreter=/usr/bin/python3

```

The inventory references the original PEM file so the playbook can connect for the first time.

5.4 The Playbook

```

---
# ssh-setup.yml
- name: Establish passwordless SSH access
  hosts: ec2_instances
  gather_facts: false
  become: false

  tasks:
    - name: Ensure the .ssh directory exists
      ansible.builtin.file:
        path: "~/ssh"
        state: directory
        mode: '0700'

    - name: Install the control node public key
      ansible.posix.authorized_key:
        user: "{{ ansible_user }}"
        state: present
        key: "{{ lookup('file', '~/ssh/id_ed25519.pub') }}"
        manage_dir: false

    - name: Verify the authorized_keys file
      ansible.builtin.stat:
        path: "~/ssh/authorized_keys"
        register: auth_keys

    - name: Report the result
      ansible.builtin.debug:
        msg: "Public key installed on {{ inventory_hostname }}"
        when: auth_keys.stat.exists

```

The `ansible.posix.authorized_key` module is the correct tool for this task. It is idempotent: if the key is already present, no change is reported. The `manage_dir: false` option prevents the module from altering the permissions of the home directory itself, which can cause problems on some AMIs.[reference:8]

5.5 Running the Playbook

```

# First run -- uses the PEM file from the inventory
ansible-playbook -i inventory/hosts.ini ssh-setup.yml

# Expected output:
# PLAY [Establish passwordless SSH access] *****
# TASK [Ensure the .ssh directory exists] *****
# ok: [web01]
# ok: [web02]
# ok: [db01]
# TASK [Install the control node public key] *****
# changed: [web01]
# changed: [web02]
# changed: [db01]
# ...

```

5.6 Verification

After the playbook runs, the PEM file is no longer required. Test the new key by connecting directly:

```
# Connect using the control node key instead of the PEM file
ssh -i ~/.ssh/id_ed25519 ec2-user@192.0.2.10

# Run an Ansible ad-hoc command without specifying the PEM file
ansible -i inventory/hosts.ini ec2_instances -m ping \
  --private-key ~/.ssh/id_ed25519
```

Once the control node key is installed on every instance, remove the `ansible_ssh_private_key_file` line from the inventory (or leave it pointing to the control node key) so that all subsequent playbooks use passwordless authentication.

6. Hardening: Disabling Password Authentication

After passwordless access is established and verified, harden the configuration by disabling password authentication entirely. This eliminates the brute-force attack surface while preserving key-based access.

```
---
# harden-ssh.yml
- name: Harden SSH configuration
  hosts: ec2_instances
  become: true

  tasks:
    - name: Disable password authentication
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PasswordAuthentication'
        line: 'PasswordAuthentication no'
        validate: '/usr/sbin/sshd -t -f %s'
        notify: Restart sshd

    - name: Disable direct root login
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PermitRootLogin'
        line: 'PermitRootLogin no'
        validate: '/usr/sbin/sshd -t -f %s'
        notify: Restart sshd

    - name: Ensure key-based authentication is enabled
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PubkeyAuthentication'
        line: 'PubkeyAuthentication yes'
        validate: '/usr/sbin/sshd -t -f %s'
        notify: Restart sshd

  handlers:
    - name: Restart sshd
      ansible.builtin.service:
        name: sshd
```

```
state: restarted
```

The `validate` parameter runs `sshd -t` against the temporary file before the change is committed. If the syntax is invalid, the original file is left untouched — a critical safety net when editing SSH configuration.

7. Best Practices

- **Prefer key-based authentication over passwords.** It is stronger, scalable, and immune to brute-force attacks.
- **Use Ed25519 keys.** They offer strong security with short key length and are supported by all modern OpenSSH versions.
- **Protect private keys with a passphrase.** Use `ssh-agent` to avoid typing it repeatedly.
- **Set PEM file permissions to 400.** OpenSSH will refuse to use a key that is readable by others.
- **Never store PEM files in the repository.** Keep them in `~/ .ssh` with restricted permissions.
- **Distribute a single control node key with Ansible.** This eliminates the need to manage individual PEM files per engineer.
- **Disable password authentication after key deployment.** Remove the fallback once passwordless access is verified.
- **Restrict SSH access by source IP.** Configure security groups to allow port 22 only from known networks.
- **Rotate keys periodically.** Generate new control node keys every 12 to 18 months and redistribute them.
- **Audit authorized_keys files.** Remove keys belonging to departed team members or decommissioned systems.
- **Use AWS Systems Manager Session Manager where possible.** It eliminates inbound SSH entirely and provides full audit logging.
- **Store secrets in Ansible Vault.** If a playbook must handle a private key, encrypt it rather than committing plain text.

8. Quick Reference

PEM File Connection

Task	Command
Set permissions	<code>chmod 400 ~/.ssh/my-key.pem</code>
Connect (Amazon Linux)	<code>ssh -i key.pem ec2-user@</code>
Connect (Ubuntu)	<code>ssh -i key.pem ubuntu@</code>
Connect (Debian)	<code>ssh -i key.pem admin@</code>
Convert to PPK (PuTTY)	<code>puttygen key.pem -o key.ppk</code>
Verify key format	<code>head -1 key.pem</code>

Table 2 — PEM file quick reference.

Password Authentication

Task	Command / File
Enable password authentication	<code>PasswordAuthentication yes</code> in <code>/etc/ssh/sshd_config</code>
Disable password authentication	<code>PasswordAuthentication no</code> in <code>/etc/ssh/sshd_config</code>
Set a user password	<code>sudo passwd</code>
Restart SSH daemon (RHEL)	<code>sudo systemctl restart sshd</code>
Restart SSH daemon (Ubuntu)	<code>sudo service ssh restart</code>
Cloud-init setting	<code>ssh_pwauth: 1</code> in <code>/etc/cloud/cloud.cfg</code>

Table 3 — Password authentication quick reference.

Ansible Passwordless Setup

Task	Command / Module
Generate control node key	<code>ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519</code>
Install public key	<code>ansible.posix.authorized_key</code>
Verify key file	<code>ansible.builtin.stat</code>
Run the playbook	<code>ansible-playbook -i inventory ssh-setup.yml</code>
Test without PEM	<code>ssh -i ~/.ssh/id_ed25519 ec2-user@</code>
Disable password auth	<code>ansible.builtin.lineinfile + validate</code>
Validate sshd config	<code>validate: /usr/sbin/sshd -t -f %s</code>

Table 4 — Ansible passwordless setup quick reference.

This document describes AWS EC2 and Ansible Core behaviour as of version 2.14 and later. Default usernames, service names, and security practices may evolve; always consult the official AWS and Ansible documentation for the version in use.